

C Programming Assignment 5 Solutions

Question 1

```
int main() {
    int a[][2][4]={5,6,7,8,9,11,12,1};
    printf("%d\n",*(*(*(a+0)+1)+2));
    return 0;
}
```

Output: 12

Question 2

```
int main() {
    int *p=NULL;
    p=(int *)malloc(sizeof(int));
    *p=10;
    free(p);
    int *q;
    q=(int *)malloc(sizeof(int));
    *q=15;
    printf("%d %d\n",*p,*q);
    return 0;
}
```

Output: Undefined 15

Pointers: Integer pointers p and q

Question 3

```
#include
#include

void fun(int **q);
int main() {
    int *p=(int *)malloc(sizeof(int));
    *p=55;
    fun(&p);
    printf("%d %p\n",*p,p);
    return 0;
}

void fun(int **q){
    int r=20;
    **q=r;
    printf("%p\n",*q);
}
```

Output:

1000

20 1000

Question 4

```
int *fun();
int main(void) {
    int *ptr;
    ptr=fun();
}
```

```

    printf("%d\n",*ptr);
    return 0;
}

int *fun() {
    int a=10,b=20;
    int sum=0;
    sum=sum+a+b;
    return &sum;
}

```

Answer: (A) Unexpected behavior
Reason: Returning address of local variable

Question 5

```

int *fun();
int main(void) {
    int *ptr=fun();
    printf("%d\n",*ptr);
    return 0;
}

int *fun() {
    int a=10,b=20,*sum;
    sum=(int *)malloc(sizeof(int));
    *sum=a+b;
    return sum;
}

```

Answer: (C) 30
Reason: malloc() allocates persistent heap memory

Question 6

```

int main() {
    int *ptr;
    ptr=(int *)realloc(NULL,sizeof(int));
    *ptr=100;
    printf("%d\n",*ptr);
    return 0;
}

```

Output: 100
Explanation: realloc(NULL,size) = malloc(size)

Question 7

```

int main() {
    int *ptr;
    ptr=(int *)calloc(1,sizeof(int));
    *ptr=100;
    printf("%d\n",*ptr);           // Line 4
    ptr=(int *)realloc(ptr,0);    // Line 6
    ptr=NULL;
    printf("%p\n",ptr);           // Line 7
    return 0;
}

```

Output at line-4: 100
Output at line-7: (nil)
Line 6 as free(): Yes

Question 8

```
int main() {
    int i;
    int *p=(int *)malloc(sizeof(int));
    *p=100;
    p=(int *)malloc(5*sizeof(int));
    for(i=0;i<5;i++) {
        scanf("%d",p+i); /* 10,20,30,40,50 */
    }
    for(i=0;i<5;i++) {
        printf("%d...%d\n",p[i],*(p+i));
    }
    return 0;
}
```

Output:

```
10...10
20...20
30...30
40...40
50...50
```

Memory: First allocation leaked, second holds 5 integers

Question 9

```
int main() {
    int i,*p,*rp;
    p=(int *)malloc(5*sizeof(int));
    for(i=0;i<5;i++)
        scanf("%d",p+i); /* 10,20,30,40,50 */
    rp=(int *)realloc(p,10*sizeof(int));
    for(i=5;i<10;i++)
        scanf("%d",rp+i); /* 9,8,6,5,4 */
    for(i=0;i<10;i++) {
        printf("%d...%d\n",rp[i],*(rp+i));
    }
    return 0;
}
```

Output:

```
10...10
20...20
30...30
40...40
50...50
9...9
8...8
6...6
5...5
4...4
```

Memory: realloc extends array from 5 to 10 elements

Question 10

```
void fun() {
    int *q=(int *)malloc(sizeof(int));
    *q=20;
}
int main() {
    int *p;
```

```
    int *r=NULL;
    fun();
    return 0;
}
```

Correct statements:

- Memory leak in fun()
- *q=20 modifies heap memory
- Local pointer q destroyed but allocated memory remains

Question 11

```
#include
#include

int main() {
    int rows=3, cols=4;
    int **arr, **transpose;
    int i,j,max,min;

    // Allocate 2D array
    arr = (int **)malloc(rows * sizeof(int *));
    for(i=0;imax) max=arr[i][j];
        if(arr[i][j]
```

Sample Output:

Enter 3x4 matrix:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

Original:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

Transpose:

```
1 5 9
2 6 10
3 7 11
4 8 12
```

Max: 12, Min: 1